KU LEUVEN

Advanced Computer Architecture

Lab Report 1

Bert Pyck
Hugo Charmant

ELICT: ELECFS
20 februari 2025
Master in Engineering Technology
Academic year 2024-2025

1.1 Task description

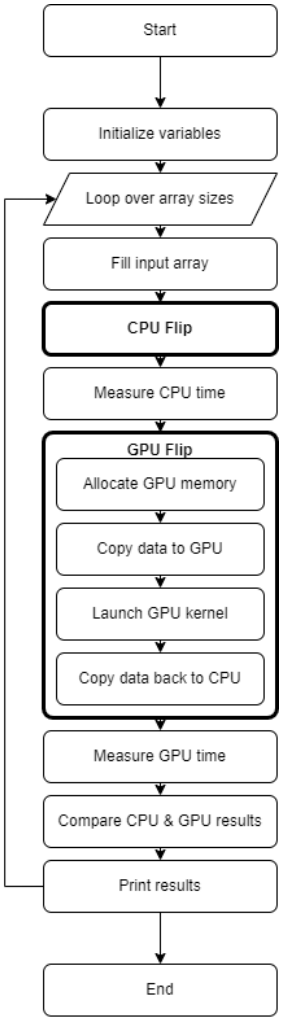
The goal of this report was to showcase our findings from the first lab session. Our task was to compute an array flip operation with the CPU and GPU to monitor differences in performance.

1.2 Introduction

We started the lab session by learning about the smallest unit of computing in a processor, the thread. Multiple threads equate to a block, and multiple blocks become a grid.
We learned that the maximal size for a block is 1024 threads, and that, as programmers, we should be careful of how we set up our threads and blocks.

1.3 Code explanation

In this chapter, the explanation of the operation and implementation of the code is given. As explained in Figure 1.1, the idea is that we need to invert the elements of an array. We will do this multiple times with the CPU and GPU. In order to achieve the accurate measurements, we will perform the operations multiple times and compute an average.



Figuur 1.1: Flowchart

1.3.1 CPU flip

This function inverts the elements of the array on the CPU *element-wise* in a simple for-loop. It copies the element from the inverted position and places it in a new array outputCPU. Before starting this for-loop, a timer will start and time measurement will take place.

With an input array of 8, the output will be as follows:

```
input: [0,1,2,3,4,5,6,7]

outputCPU[i] = input[(SIZE-1) - i];

output: [7,6,5,4,3,2,1,0]
```

1.3.2 GPU flip

This function will run differently. It will use CUDA to perform the operation on the GPU. Each thread will process one element of the array. If the array is of a size greater than 1024, this means that we can't use a single block. In our code, if we detect an array length

greater than 1024, we created additional blocks to fit the full array. Next, CUDA memory will be allocated. After that, data will be copied to the GPU, and the kernel will be started **gpu_flip<<<numBlocks, BLOCK_SIZE>>>**. The kernel will calculate the thread index and reverse index mirror. If the thread index falls within the array bounds it performs the flip. When the kernel has done its job the results will be copied back to the CPU and the timing will be stopped. The operation here is parallel and this is due to each GPU thread working on one element, which means we utilize the strength of the GPU.

With an input array of 8, the output will be as follows:

```
input (host): [0,1,2,3,4,5,6,7]
GPU memory : [0,1,2,3,4,5,6,7]

Thread t_id mirror operation
    0 0 7 d_output[0] = d_input[7]
    1 1 6 d_output[1] = d_input[6]
    2 2 5 d_output[2] = d_input[5]
    3 3 4 d_output[3] = d_input[4]
    4 4 3 d_output[4] = d_input[3]
    5 5 2 d_output[5] = d_input[2]
    6 6 1 d_output[6] = d_input[1]
    7 7 0 d_output[7] = d_input[0]

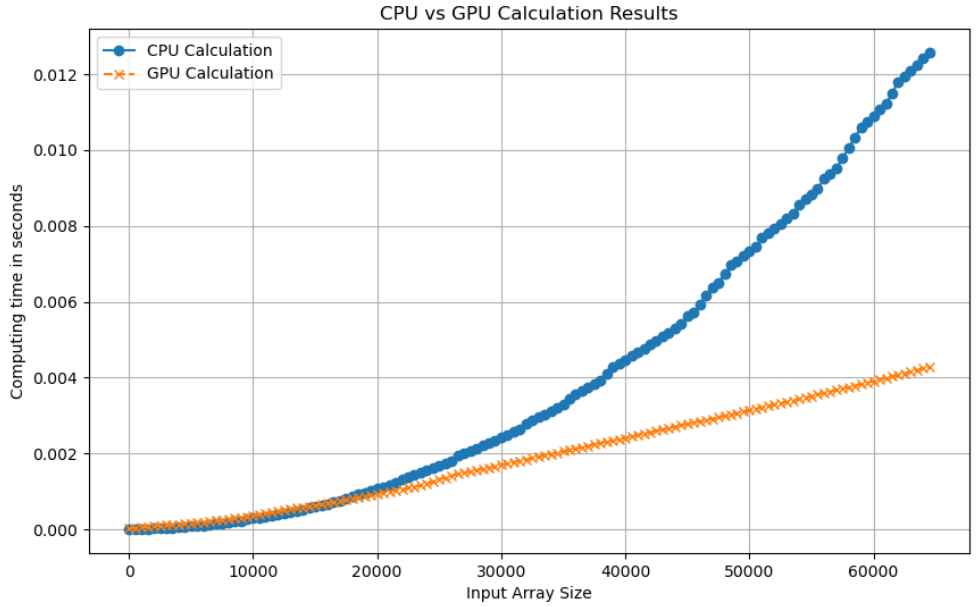
output (host): [7,6,5,4,3,2,1,0]
```

In the results, we saw that the computing time was not constant, sometimes even taking twice as long for the same operation. Following Mr. Asefaw's detailed explanation, we discovered that the machine was running additional processes, meaning that the difference in computing time was normal. In order to eliminate this variance, we performed an average, where we took 1000 measurements and used the average time for comparison. Performing this calculation for multiple array sizes allowed us to see how both computations scale. This is the reason for the additional blocks in the flowchart in Figure 1.1 meant to increase the array size and to display and compare time values.

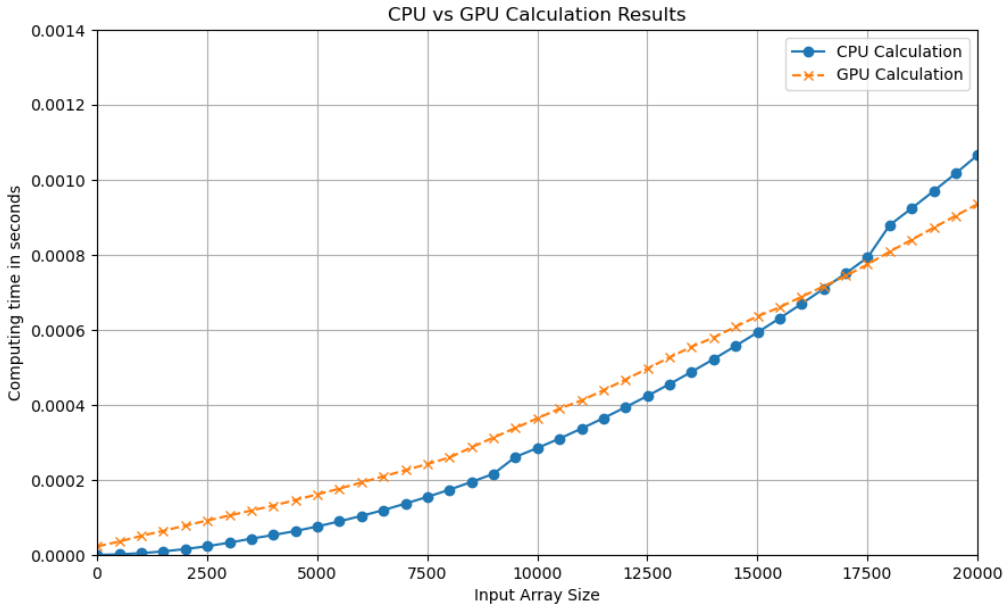
1.4 Results and conclusion

The results of our measurements can be found in figure 1.2 and figure 1.3. From these graphs, it is clear that the GPU exponentially outperforms the CPU when the array grows in size. Looking closer at the smaller array sizes, we can see that the CPU still outperforms the GPU at smaller sizes. This is due to the overhead needed to transfer data from the CPU to the GPU. The GPU outperforms the CPU in situations where parallel processing is needed, like in our function *GPU_flip*.

We therefore conclude that the use of a GPU for calculations may improve the calculation time, only when the operation utilizes parallelism, and, when the calculation is 'heavy' enough to account for the data transfer time from CPU to GPU.



Figuur 1.2: Comparison of computing time for an array flipping operation, CPU vs GPU



Figuur 1.3: Close up, showcasing the part where the CPU is faster than the GPU

2 CODE

```
#include <iostream>
#include <cuda_runtime.h>
#include <chrono>

#define BLOCK_SIZE 1024
#define AVERAGE 1000

using namespace std;

unsigned long int SIZE;

double resultCPU = 0;
double resultGPU = 0;

__host__ double cpu_flip(unsigned long int outputCPU[], unsigned long int input
    []) {
    const auto start = std::chrono::steady_clock::now();

    for (int i = 0; i < SIZE; i++) {
        outputCPU[i] = input[(SIZE-1) - i];
    }

    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds = end - start;
    double timeCount = elapsed_seconds.count();

    return timeCount;
}

__global__ void gpu_flip(unsigned long int *d_input,unsigned long int *d_output,
    int size) {
    int t_id = threadIdx.x + blockIdx.x * blockDim.x;
    int mirror = size - 1 - t_id;

    if (t_id < size) {
        d_output[t_id] = d_input[mirror];
    }
}

int main(int argc, char **argv) {
    std::cout << "SIZE;" << "CPU;" << "GPU" << std::endl;
    for(int j = 8; j < 65000; j=j+500){
        SIZE = j;
        unsigned long int input[SIZE];
        unsigned long int outputCPU[SIZE];
```

```
44          unsigned long int outputGPU[SIZE];
45          // Creates an array of size size
46
47          for (int i = 0; i < SIZE ; i++){
48              input[i] = i;
49          }
50
51          // CPU calculation
52          for(int i = 0; i < AVERAGE; i++){
53              resultCPU = resultCPU + cpu_flip(outputCPU, input);
54          }
55
56          for(int i = 0; i < AVERAGE; i++){
57              cudaEvent_t start_cuda, stop_cuda;
58              cudaEventCreate(&start_cuda);
59              cudaEventCreate(&stop_cuda);
60
61
62          // Allocate memory on GPU
63              unsigned long int *d_input, *d_output;
64              cudaMalloc(&d_input, SIZE * sizeof(unsigned long int));
65              cudaMalloc(&d_output,
66              SIZE * sizeof(unsigned long int));
67
68              // Copy data from host to device
69              cudaMemcpy(d_input, input, SIZE * sizeof(unsigned long int),
                    cudaMemcpyHostToDevice);
70
71              cudaEventRecord(start_cuda);
72              // Launch kernel (calculate required blocks)
73              int numBlocks = (SIZE + BLOCK_SIZE - 1) / BLOCK_SIZE;
74              gpu_flip<<<numBlocks, BLOCK_SIZE>>>(d_input, d_output, SIZE);
75
76              cudaDeviceSynchronize();
77              cudaEventRecord(stop_cuda);
78              cudaEventSynchronize(stop_cuda);
79
80              // Copy result back to host
81              cudaMemcpy(outputGPU, d_output, SIZE * sizeof(unsigned long int),
                    cudaMemcpyDeviceToHost);
82
83              // Free device memory
84              cudaFree(d_input);
85              cudaFree(d_output);
86
87
88              float ms;
89              cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
90              resultGPU += ms/1000;
91          }
92
93          bool match = true;
94          for (int i = 0; i < SIZE; i++) {
95              if (outputCPU[i] != outputGPU[i]) {
96                  match = false;
97                  break;
```

```
98              }
99          }
100     }
101     return 0;
102 }
```

Listing 2.1: The array flipping Cuda code